Add 算子 examples 测试设计说明

1. 测试目标与策略

本测试文件采用 **examples 风格的端到端测试**，目标是验证 aclnnAdd / aclnnAdds / aclnnInplaceAdd / aclnnInplaceAdds / aclnnAddV3 在 CPU 模拟器或实际运行环境下的**执行正确性**与**参数校验行为**。

测试策略分为两类：

1. **正常功能用例**
   - 通过真实调用 ACLNN 两段式接口：
     - GetWorkspaceSize
     - Execute
   - 执行后将结果从设备拷回主机
   - 使用 CPU 侧独立计算的期望值逐元素比对
   - 用于验证数值正确性，并覆盖主流程分支
2. **异常/非法输入用例**
   - 构造空指针、shape 不可广播、输出 shape 错误、dtype 不支持、dtype 不匹配等场景
   - 预期 GetWorkspaceSize 返回失败
   - 用于覆盖 op_api 层参数校验分支

之所以采用上述策略，是因为：

- examples 测试比 UT 更贴近真实执行链路；
- 能同时覆盖 **参数校验、类型处理、shape 推断、workspace 规划、执行与结果校验**；
- 对精度问题和真实运行问题更敏感。

---

2. 用例设计思路

2.1 API 维度覆盖

本测试覆盖了 Add 相关的 5 类接口：

- aclnnAdd：tensor + tensor
- aclnnAdds：tensor + scalar
- aclnnInplaceAdd：inplace tensor + tensor
- aclnnInplaceAdds：inplace tensor + scalar
- aclnnAddV3：scalar + tensor

设计目标：

- 覆盖不同 API 在 op_api 层的不同调度逻辑
- 覆盖普通输出与原地写回两类路径
- 覆盖 scalar/tensor 混合输入路径

2.2 dtype 覆盖

测试覆盖了以下 dtype：

- ACL_FLOAT
- ACL_FLOAT16
- ACL_BF16
- ACL_DOUBLE
- ACL_INT8
- ACL_UINT8

- ACL_INT16
- ACL_INT32
- ACL_INT64
- ACL_BOOL

同时覆盖了若干 **混合 dtype** 场景，例如：

- FLOAT16 + FLOAT -> FLOAT
- FLOAT + BF16 -> FLOAT

设计目标：

- 覆盖常见浮点、整数、布尔类型
- 触发类型提升/类型转换相关路径
- 触发不同 dtype 的 host/kernel 调度分支
- 覆盖低精度类型（FP16/BF16）的精度判断逻辑

2.3 shape 覆盖

测试覆盖的 shape 类型包括：

1. **同 shape**
   - 如 {4,2} + {4,2}
   - 覆盖最基本逐元素计算路径
2. **广播 shape**
   - 如 {2,3} + {3}
   - 如 {2,3} + {1,3}
   - 覆盖广播推断逻辑与对应执行路径
3. **空 tensor**
   - 如 {0,3}
   - 覆盖零元素场景，检查边界下是否能正常创建和执行
4. **大 tensor**
   - 如 {32,64}
   - 目标是尽量触发更明显的 workspace / tiling 路径
5. **非连续内存布局**
   - 使用自定义 stride 构造 non-contiguous tensor
   - 覆盖连续/非连续内存处理差异

设计目标：

- 覆盖 infershape 中的广播逻辑
- 覆盖普通 shape 与大 shape 路径
- 覆盖空 shape / 边界 shape
- 尽量覆盖 stride/layout 相关分支

2.4 数值与边界条件覆盖

测试中覆盖了以下数值场景：

- 正数、负数、零
- alpha = 0
- alpha < 0
- 大小不同的整数值
- double 特殊值：

    - NaN
    - Inf
    - -Inf

设计目标：

- 验证 self + alpha * other 在不同 alpha 下的正确性
- 检查浮点特殊值传播行为
- 检查整数类型的转换和截断表现
- 检查布尔类型按非零/零语义处理的行为

---

3. 期望值与精度校验设计

3.1 CPU 参考结果

所有正常用例均在 CPU 侧独立计算期望值：

- tensor-tensor：按广播规则逐元素计算
- tensor-scalar：逐元素加标量
- scalar-tensor：逐元素加上 scalar 输入
- inplace：以原输入 shape/type 计算期望写回结果

3.2 精度容差

根据输出 dtype 设置不同容差：

- BF16：1e-2
- FLOAT16：1e-3
- FLOAT：1e-5
- DOUBLE：1e-12
- 整数/布尔：精确比较

设计依据：

- 不同 dtype 的有效精度不同；
- examples 测试应避免因正常浮点舍入误差误报失败；
- 同时保留对真实精度异常的敏感性。

---

4. 异常用例设计目标

异常用例主要覆盖参数校验逻辑，包含：

4.1 空指针类

- self == nullptr
- other == nullptr
- alpha == nullptr
- out == nullptr

对应接口包括：

- Add
- Adds
- InplaceAdd
- InplaceAdds
- AddV3

4.2 shape 错误类

- 输入 shape 不可广播
- 输出 shape 与推导结果不匹配

4.3 dtype 错误类

- 不支持 dtype：UINT32
- 输出 dtype 不匹配
- 非法混合 dtype：如 FLOAT + INT32

设计目标：

- 覆盖 op_api 层最关键的参数合法性检查分支
- 验证接口能否在错误输入下返回失败而非静默执行

---

5. 覆盖率分析

5.1 已覆盖内容

从设计上看，测试重点覆盖了以下代码区域：

op_api 层

- 各 API 的 GetWorkspaceSize 主流程
- 正常参数校验分支
- 异常参数校验分支
- dtype 组合处理分支
- inplace / scalar / tensor 不同调度路径
- 非连续 tensor 的相关路径

op_host 层

- 广播 shape 推断逻辑
- 不同 shape 下的输出推导
- 大 tensor 场景下更可能触发的 tiling/workspace 规划路径

examples 侧自定义逻辑

- 数据准备
- tensor/scalar 创建
- 结果回拷
- CPU 参考计算
- 精度比较与错误日志输出

5.2 未覆盖或覆盖有限的内容

1. kernel 层真实覆盖不可观测

在 CPU 模拟器环境下，设备侧 kernel 执行通常无法通过 gcov 直接插桩，因此：

- 即使实际执行了 kernel
- 也不一定能在覆盖率报告中看到对应行/分支覆盖

2. 部分分支依赖运行环境

如：

- 特定硬件能力分支
- 自定义算子库动态加载路径
- 某些平台专属路由逻辑

如果环境缺少依赖库或未启用对应硬件特性，相关分支不会被覆盖。

3. 某些 dtype 组合仍未完全穷举

虽然覆盖了多种主流和典型混合类型，但并未对所有注册支持组合进行全枚举，因此：

- 个别少见 dtype 路径可能未触发
- 某些稀有分支覆盖仍有限

4. 异常用例在部分环境下可能不稳定

空指针类用例理论上用于覆盖参数校验分支，但在某些模拟器/运行时环境中，底层可能直接崩溃而非安全返回错误码，因此这类分支覆盖可能受环境限制。

---

6. 总结

本测试的核心思路是：

- 以 **端到端执行正确性** 为主线；
- 结合 **CPU 参考计算 + 容差校验** 保证结果有效性；
- 通过 **多 API、多 dtype、多 shape、多边界值** 提升主流程覆盖；
- 通过 **空指针、shape 错误、dtype 错误** 覆盖关键参数校验分支。

整体上，该设计能够较好覆盖 Add 算子的：

- 正常执行链路
- 广播与 shape 推断逻辑
- 类型分发与部分类型提升路径
- inplace/scalar 变体路径
- 关键异常分支

未覆盖部分主要受限于：

- 运行环境依赖
- kernel 层不可插桩
- 平台相关分支不可稳定触发